LONG/SHORT FACTORY — AUTONOMOUS OVERNIGHT IMPLEMENTATION GOAL

You are the lead engineer, workflow architect, QA engineer, migration engineer and AI integration engineer for the Long/Short Factory repository.

You are authorized to work autonomously through the night inside this repository.

Do not pause for routine implementation approval.

Do not ask the user to approve every batch.

Do not wait between safe checkpoints.

You may:

- inspect code;
- run CodeGraph;
- edit repository files;
- add focused dependencies when necessary;
- create SQLite migrations;
- run tests;
- launch Electron;
- run local provider capability checks;
- call the already configured 9Router gateway;
- create local test projects;
- generate test text and test images;
- create commits after safe checkpoints;
- continue to the next independent checkpoint;
- write complete issue and audit reports.

You must not:

- modify files outside the repository or configured workspace;
- delete user projects or generated assets without an explicit backup;
- expose provider credentials;
- bypass workflow gates;
- mark failed outputs as approved;
- fabricate successful provider results;
- replace real execution with hidden fixtures;
- silently skip validation;
- create a global one-click `Generate Full Project` workflow;
- automatically approve editorial outputs for the user;
- rewrite the entire architecture without evidence;

- perform broad unrelated refactoring.

The objective is not to make every menu item appear functional.

The objective is to build a correct, auditable and stage-gated YouTube production workflow.

---

1. OPERATING PRINCIPLE

The application must follow:

```
Input
→ Validation
→ Explicit Run
→ Execution
→ Structured Output
→ Validation
→ Human Review
→ Approve or Reject
→ Unlock Next Stage
```

The application must never follow:

```
Topic
→ Generate
→ Entire project automatically appears
```

No downstream production stage may consume:

- draft input;
- rejected input;
- failed output;
- stale output;
- unverified provider capability;
- unapproved upstream artifact.

Downstream stages may consume only approved upstream artifacts.

---

2. AUTONOMOUS EXECUTION POLICY

Work continuously without waiting for user responses.

For each checkpoint:

```
1. Inspect current implementation.
2. Confirm upstream contracts.
3. Run focused CodeGraph exploration.
4. Define the smallest coherent change.
5. Implement.
6. Run automated tests.
7. Run Electron/runtime verification where possible.
8. Audit persistence and security.
9. Classify discovered issues.
10. Record issues in Markdown and JSON.
11. Commit safe work.
12. Continue to the next valid checkpoint.
```

Do not stop for P1, P2 or P3 issues when independent work remains possible.

Stop the entire overnight run only for P0 conditions described later.

---

3. STARTUP AUDIT

Begin by running:

```
git status
git branch --show-current
git log -10 --oneline
codegraph status

corepack pnpm lint
corepack pnpm typecheck
corepack pnpm test
corepack pnpm --filter @lsf/desktop build
```

Record results before changing code.

Create:

```
docs/overnight/OVERNIGHT_RUN_2026-07-30.md
docs/overnight/OPEN_ISSUES_2026-07-30.md
docs/overnight/overnight-issues-2026-07-30.json
docs/overnight/OVERNIGHT_CHECKPOINTS_2026-07-30.md
```

If the files already exist, append a new timestamped run section rather than deleting history.

Record:

- starting commit;
- branch;
- dirty files;
- Node version;
- pnpm version;
- Electron version;
- Python sidecar path;
- pycapcut version;
- FFmpeg path/version;
- workspace path;
- SQLite path;
- 9Router base URL without credentials;
- selected models;
- capability certification states.

Never write the raw API key.

---

4. EXISTING PROVIDER ASSUMPTIONS

9Router is already configured by the user.

Do not rebuild account management.

Treat the following as separate facts:

```
Credential saved
Endpoint reachable
Model discovered
Model selected
Capability verified
```

A discovered or selected model is not automatically verified.

Use only a model whose exact capability certification is currently `verified`.

Certification fingerprint must include:

- provider ID;
- base URL fingerprint;
- credential version reference;

- model ID;
- endpoint strategy;
- certification implementation version.

When any relevant value changes, mark certification `stale`.

Do not infer capabilities from model names.

Do not silently switch:

- provider;
- model;
- endpoint;
- request format.

If a configured capability is missing or stale:

1. run the existing certification workflow;
2. validate it;
3. persist certification;
4. continue only after verified.

9Router limits are already controlled externally by the user.

Do not add arbitrary low request limits.

Still prevent:

- infinite retry;
- duplicate request loops;
- uncontrolled recursion;
- repeated paid calls caused by renderer rerenders;
- automatic generation on page load;
- generation on application restart.

Every provider-backed run must have a unique idempotency/input fingerprint and persisted execution record.

---

5. ISSUE SEVERITY POLICY

P0 — Stop the entire overnight run

Examples:

- raw credential written to SQLite, log, renderer or export;

- database corruption;
- migration causes data loss;
- code deletes files outside workspace;
- uncontrolled paid-call loop;
- duplicate provider requests cannot be stopped;
- project data cannot be restored after migration;
- arbitrary model output is executed as code or shell;
- renderer can invoke arbitrary filesystem or runner operations;
- security boundary is disabled;
- Git destructive operation would lose uncommitted user work.

P0 action:

```
Stop all code changes
→ preserve repository state
→ write complete evidence
→ do not commit broken changes
→ leave recovery instructions
```

P1 — Block the dependent workflow, continue independent work

Examples:

- provider stage fails;
- invalid provider response;
- output fails Zod validation;
- image cannot be decoded;
- stage persistence fails;
- approval logic is incorrect;
- duplicate reference remains unresolved;
- restart loses a StageRun;
- stale invalidation is incorrect;
- FFmpeg preview fails;
- CapCut draft cannot open.

P1 action:

```
Mark stage Failed or Blocked
→ keep downstream blocked
→ write issue
→ continue independent audit and implementation
```

P2 — Continue and record

Examples:

- non-critical UI issue;
- unclear text;
- missing loading detail;
- history table missing optional metadata;
- visual alignment problem;
- non-destructive edge case.

P3 — Backlog improvement

Examples:

- wording;
- polish;
- optional filters;
- component cleanup;
- nonessential optimization.

Never treat P1–P3 as success.

Never fake output to bypass a failed stage.

---

6. REQUIRED ISSUE FORMAT

Every issue must be written to both:

```
docs/overnight/OPEN_ISSUES_2026-07-30.md
docs/overnight/overnight-issues-2026-07-30.json
```

Markdown format:

```
## ISSUE-XXX — Clear title

- Severity: P0 / P1 / P2 / P3
- Status: Open
- Detected at:
- Checkpoint:
- Project ID:
- Stage ID:
- StageRun ID:
```

```
- Component:
- Related files:
- Reproducible: Yes / No / Unknown
- Blocks downstream: Yes / No
- Safe independent work remains: Yes / No

### Reproduction

1.
2.
3.

### Expected

...

### Actual

...

### Evidence

- Screenshot:
- Log:
- Database query:
- Test:
- Commit:

### Root-cause analysis

Confirmed / suspected.

### Safety action taken

...

### Suggested fix batch

...

### Acceptance criteria

...
```

JSON item format:

```
{
  "id": "ISSUE-001",
  "severity": "P1",
  "status": "open",
  "checkpoint": "reference-intake",
  "projectId": null,
  "stageId": "reference-validation",
  "stageRunId": null,
  "title": "",
  "expected": "",
  "actual": "",
  "reproduction": [],
  "evidence": [],
  "blocksDownstream": true,
  "safeIndependentWorkRemains": true,
  "suggestedFixBatch": "",
  "acceptanceCriteria": []
}
```

Never omit an issue merely because it was fixed later.

When fixed, update:

```
Status: Fixed
Fixed by commit:
Verification evidence:
```

---

7. CANONICAL PRODUCTION WORKFLOW

Use this workflow as the source of truth:

```
01. Project Setup
02. Reference Intake
03. Reference Validation
04. Transcript Cleaning
05. Reference Segmentation
06. Competitor DNA
07. Opportunity Map
08. Idea Lab
09. Originality Review
10. Research Source Intake
11. Claim Map
```

```
12. Outline
13. Script
14. Fact Review
15. Retention Review
16. Scene Plan
17. Shot Plan
18. Visual Routing
19. Prompt Preparation
20. Asset Acquisition
21. Asset Review
22. Voice Generation
23. Subtitle Preparation
24. Timeline Assembly
25. Preview Render
26. QA
27. CapCut Draft
28. Packaging Export
```

Do not automatically collapse these stages into one provider call.

Screens may group stages visually, but every stage retains:

- its own eligibility;
- StageRun history;
- runner;
- input fingerprint;
- output artifact;
- validation;
- review;
- approval;
- stale state.

---

8. CENTRAL WORKFLOW CONTRACT

Create or update:

```
docs/WORKFLOW_CONTRACT.md
docs/WORKFLOW_STAGE_MATRIX.md
docs/SCREEN_STATE_CONTRACT.md
docs/CODEX_BATCH_RULES.md
```

Implement a trusted domain-level registry:

```ts
interface WorkflowStageDefinition {
  id: string;
  name: string;
  order: number;
  route: string;

  dependsOn: string[];

  requiredInputTypes: string[];
  outputArtifactTypes: string[];

  runnerId: string | null;

  executionKind:
    | "manual_input"
    | "local_deterministic"
    | "provider_text"
    | "provider_image"
    | "provider_video"
    | "provider_audio"
    | "media_process"
    | "export";

  requiredCapability?: string;

  approvalRequired: boolean;
  invalidates: string[];
}
```

Negative checks:

- no duplicate stage IDs;
- no missing dependency ID;
- no cycle;
- no invalid invalidation target;
- stable stage order;
- every stage has a route;
- every provider stage declares a capability;
- every output type is registered;
- renderer must not maintain a separate dependency graph.

---

9. CENTRAL STAGE STATE

Use one vocabulary:

```ts
type WorkflowStageStatus =
  | "not_started"
  | "blocked"
  | "ready"
  | "queued"
  | "running"
  | "needs_review"
  | "approved"
  | "rejected"
  | "failed"
  | "stale";
```

Allowed run transitions:

```
queued → running
running → needs_review
running → failed
needs_review → approved
needs_review → rejected
approved → stale
```

A rerun creates a new StageRun.

Forbidden:

```
running → approved
failed → approved
rejected → approved
stale → approved automatically
approved → running on the same run
```

Renderer cannot directly set status.

All transitions occur through a trusted main-process service.

---

10. ELIGIBILITY CONTRACT

Every stage must provide:

```ts
interface StageEligibility {
  stageId: string;
  status: WorkflowStageStatus;

  viewable: boolean;
  runnable: boolean;
  reviewable: boolean;
  approvable: boolean;

  blockingReasons: Array<{
    code: string;
    message: string;
    actionRoute?: string;
  }>;
}
```

Negative checks:

- a screen may be viewable while blocked;
- blocked is not runnable;
- needs_review is not runnable unless explicit rerun is used;
- approved does not automatically rerun;
- provider stage requires verified capability;
- missing approved dependency blocks the stage;
- stale upstream blocks downstream;
- unresolved duplicate blocks reference-set approval;
- invalid included reference blocks approval;
- disabled actions require actionable explanations.

Do not show an empty stage screen with only a disabled button.

---

11. SHARED STAGE SCREEN STRUCTURE

Every production screen must show:

```
Stage number and name
Purpose
Current status
Dependencies
Eligibility
Blocking reasons
Current approved input
Provider/model requirement
Run controls
```

```
Latest StageRun
Input fingerprint
Structured output
Validation result
Review controls
Run history
Downstream impact
```

Reusable components should include:

```
StageHeader
StageStatusPanel
EligibilityPanel
BlockingReasonPanel
CurrentInputPanel
RunControlPanel
RunMetadataPanel
OutputReviewPanel
ApprovalPanel
RunHistoryPanel
DownstreamImpactPanel
```

Do not redesign unrelated UI.

Do not add another global state library unless demonstrably required.

---

12. CHECKPOINT A — REFERENCE INTAKE

Implement a complete Reference Intake workflow.

Reference states:

```ts
type ReferenceStatus =
  | "draft"
  | "duplicate"
  | "invalid"
  | "valid"
  | "approved"
  | "rejected"
  | "stale";
```

Each reference supports:

```
View
Edit
Delete
Validate
Include for analysis
Exclude from analysis
View versions
Replace transcript
```

Reference set supports:

```
Validate all
Review warnings
Approve reference set
Revoke approval
```

A saved reference is not automatically approved.

URL identity

Canonicalize YouTube URLs by video ID.

Treat these as identical:

```
https://www.youtube.com/watch?v=VIDEO_ID
http://youtube.com/watch?v=VIDEO_ID
https://youtu.be/VIDEO_ID
https://m.youtube.com/watch?v=VIDEO_ID
```

Ignore:

- protocol;
- www/mobile hostname;
- timestamp;
- playlist;
- tracking parameters.

Do not silently create duplicate rows.

Duplicate UI:

```
This source already exists.
```

```
Open existing
Replace transcript
Create new transcript version
Cancel
```

A transcript version belongs to the existing source identity.

Validation

Validate:

- source identity;
- URL shape when present;
- transcript non-whitespace;
- transcript length;
- include/exclude state;
- unresolved duplicate;
- required content.

Reference-set approval requires:

```
At least one included valid reference
No unresolved included duplicate
No invalid included reference
Validation completed for current fingerprint
```

Negative checks

- `references.length > 0` is not sufficient;
- duplicate URL cannot bypass by HTTP/HTTPS variation;
- excluded invalid reference does not block approval;
- included invalid reference blocks approval;
- changing approved reference marks approval stale;
- deleting approved reference marks downstream stale;
- editing notes alone only invalidates stages if notes are part of approved analysis input;
- old versions remain visible;
- no automatic provider call.

Commit after this checkpoint if safe.

---

13. CHECKPOINT B — REFERENCE VALIDATION STAGE ENGINE

Complete the local deterministic reference validation StageRun.

Persist:

```
interface WorkflowStageRun {
  id: string;
  projectId: string;
  stageId: string;

  status: WorkflowStageStatus;

  runnerId: string;
  runnerVersion: string;

  providerId?: string;
  configuredModelId?: string;
  returnedModelId?: string;

  promptTemplateId?: string;
  promptVersion?: string;

  inputArtifactIds: string[];
  inputFingerprint: string;

  outputArtifactIds: string[];

  startedAt?: string;
  finishedAt?: string;

  safeErrorCategory?: string;
  safeErrorMessage?: string;
}
```

Persist output artifact:

```
interface WorkflowArtifact {
  id: string;
  projectId: string;
  stageId: string;
  stageRunId: string;
```

```
  type: string;
  version: number;

  status:
    | "draft"
    | "needs_review"
    | "approved"
    | "rejected"
    | "stale";

  payloadJson?: Record<string, unknown>;
  relativeFilePath?: string;

  createdAt: string;
  updatedAt: string;
}
```

Input fingerprint uses canonical SHA-256 JSON.

Exclude:

- secrets;
- timestamps;
- absolute paths;
- temporary URLs.

Negative checks:

- duplicate double-click cannot create two active runs;
- old runs are never overwritten;
- rejected output remains;
- approval occurs transactionally;
- restart retains history;
- edit approved input marks previous run/artifact stale;
- no downstream automatic run.

Commit after this checkpoint if safe.

---

14. CHECKPOINT C — TRANSCRIPT CLEANING

Implement Transcript Cleaning as the first production text stage.

Inputs:

- approved reference set;
- verified text model;
- selected included references;
- raw transcript version;
- channel profile rules.

For each included reference, create a separate cleaning run and cleaned transcript artifact.

Do not combine multiple references into one transcript.

The cleaner may:

- normalize whitespace;
- remove obvious timestamps when requested;
- remove duplicated caption fragments;
- preserve semantic content;
- preserve uncertain wording;
- flag unintelligible fragments;
- preserve quotations.

It must not:

- summarize;
- rewrite story structure;
- add facts;
- remove claims merely because they seem wrong;
- translate unless project explicitly requests translation;
- perform Competitor DNA;
- generate ideas.

Output strict JSON, validated by Zod:

```
{
  referenceId: string;
  sourceTranscriptVersionId: string;
  cleanedTranscript: string;
  removedSegments: Array<{
    text: string;
    reason:
      | "timestamp"
      | "duplicate_caption"
      | "formatting_noise"
      | "empty_fragment";
  }>;
  flaggedSegments: Array<{
```

```
    text: string;
    reason:
      | "unclear_audio"
      | "possible_transcription_error"
      | "language_mismatch";
  }>;
  sourceCharacterCount: number;
  cleanedCharacterCount: number;
}
```

UI must show a diff/review view.

Approval is per reference.

Negative checks:

- model output not strict JSON → failed;
- missing transcript → blocked;
- output empty when input nonempty → failed;
- excessive content loss warning;
- content expansion warning;
- model must not add unrelated paragraphs;
- raw transcript remains immutable;
- failed cleaning cannot unlock segmentation;
- rerun creates a new version;
- no automatic approval.

Continue even if one reference fails; keep that reference and dependent aggregate stages blocked while auditing other independent references.

Commit if safe.

---

15. CHECKPOINT D — REFERENCE SEGMENTATION

Run only on approved cleaned transcripts.

Per-reference structured output:

```
{
  referenceId: string;
  segments: Array<{
    id: string;
    order: number;
    startCharacter: number;
```

```
    endCharacter: number;

    type:
      | "hook"
      | "promise"
      | "context"
      | "problem"
      | "conflict"
      | "evidence"
      | "example"
      | "reveal"
      | "payoff"
      | "cta"
      | "other";

    text: string;
    function: string;
  }>;
}
```

Negative checks:

- segments ordered;
- no overlap unless explicitly allowed;
- no character range outside transcript;
- reconstructed segment text must correspond to cleaned transcript;
- no invented segment text;
- at least one segment;
- no Competitor DNA analysis yet;
- approval per reference;
- stale when cleaned transcript changes.

Commit if safe.

---

16. CHECKPOINT E — COMPETITOR DNA

Run separately for each reference after that reference has approved segmentation.

Input must include:

- approved cleaned transcript;
- approved segments;
- channel profile;
- originality rules.

Output strict structured JSON:

```
{
  referenceId: string;

  hookPattern: {
    abstraction: string;
    evidenceSegmentIds: string[];
  };

  promisePattern: {
    abstraction: string;
    evidenceSegmentIds: string[];
  };

  pacingPattern: {
    description: string;
    evidenceSegmentIds: string[];
  };

  proofPattern: {
    description: string;
    evidenceSegmentIds: string[];
  };

  emotionalArc: Array<{
    phase: string;
    emotion: string;
    evidenceSegmentIds: string[];
  }>;

  retentionDevices: Array<{
    abstraction: string;
    evidenceSegmentIds: string[];
  }>;

  visualOpportunities: Array<{
    abstraction: string;
    evidenceSegmentIds: string[];
  }>;

  reusablePrinciples: string[];

  forbiddenToCopy: Array<{
    element: string;
    reason: string;
  }>;
```

```
  uncertainties: string[];
}
```

The stage analyzes abstractions, not reusable wording.

Negative checks:

- no copied long phrases;
- no title copying;
- no thumbnail copying;
- every important finding references evidence segments;
- no invented evidence;
- no analysis from unapproved transcripts;
- no combining outputs across references;
- no automatic Opportunity Map run;
- output must pass originality phrase-overlap checks.

Commit if safe.

---

17. CHECKPOINT F — OPPORTUNITY MAP

Runs only when the configured minimum number of Competitor DNA outputs are approved.

Use approved DNA artifacts, not raw transcripts.

Output:

```
{
  sharedPatterns: [],
  overusedPatterns: [],
  underservedViewerQuestions: [],
  evidenceGaps: [],
  differentiationDirections: [],
  riskyDirections: [],
  recommendedContentSpaces: []
}
```

Each item should cite source reference IDs or DNA artifact IDs.

Negative checks:

- no raw competitor wording;

- no unsupported market-size claims;
- no claim that a direction is unique without evidence;
- one reference should not be presented as industry consensus;
- if only one reference exists, mark confidence lower;
- no automatic Idea Lab run;
- approval required.

Commit if safe.

---

18. CHECKPOINT G — IDEA LAB

Runs only from:

- approved Opportunity Map;
- active channel profile;
- project format;
- language;
- target duration.

Generate 12 candidates by default:

```
4 low-risk evergreen
4 medium-difficulty
4 stretch
```

Output per idea:

```
{
  id: string;
  workingTitle: string;
  angle: string;
  corePromise: string;
  viewerProblem: string;
  dramaticQuestion: string;
  targetEmotion: string;
  trafficModel: "search" | "browse" | "mixed";
  thumbnailConcept: string;
  noveltyExplanation: string;
  noveltyScore: number;
  audienceFitScore: number;
  thumbnailPotentialScore: number;
  researchRisk: "low" | "medium" | "high";
  productionDifficulty: "low" | "medium" | "high";
```

```
  whyThisCanWin: string;
}
```

User must:

- edit;
- reject;
- shortlist;
- approve exactly one idea.

Negative checks:

- no deterministic fixture pretending to be AI;
- no copied competitor title;
- no exact thumbnail reuse;
- no impossible promise;
- no medical/legal/financial guarantee;
- no defamation;
- no auto-selecting an idea;
- no auto-running research or script;
- approved idea change invalidates downstream stages.

Commit if safe.

---

19. CHECKPOINT H — ORIGINALITY REVIEW

Compare approved idea against:

- competitor titles;
- repeated phrases;
- hook patterns;
- promise sequence;
- thumbnail concepts;
- channel's own stored ideas when available.

Output:

```
{
  phraseOverlapRisk: number;
  structuralOverlapRisk: number;
  thumbnailOverlapRisk: number;
  conceptOverlapRisk: number;
  flaggedMatches: [];
  requiredChanges: [];
```

```
  status: "pass" | "needs_changes" | "blocked";
}
```

Negative checks:

- do not use only string similarity;
- distinguish common niche terminology from copied phrasing;
- blocked originality cannot proceed;
- changing idea requires rerun;
- no automatic idea rewrite unless user explicitly runs a revision action.

Commit if safe.

---

20. CHECKPOINT I — RESEARCH SOURCE INTAKE AND CLAIM MAP

Do not fabricate web research.

When automatic source retrieval is not implemented, create a correct manual source intake and claim-map workflow.

Sources have:

```
Source URL
Title
Publisher
Published date
Source type
Primary/secondary
Notes
Local snapshot/reference
```

Claims have:

```
{
  id: string;
  text: string;
  type:
    | "fact"
    | "interpretation"
    | "allegation"
    | "opinion"
```

```
    | "historical_context";

  sourceRequirement: "primary" | "secondary" | "either";
  sourceIds: string[];

  confidence: number;

  state:
    | "unsupported"
    | "partially_supported"
    | "supported"
    | "contradicted"
    | "needs_qualification";

  qualification?: string;
}
```

Negative checks:

- no unsupported claim marked supported;
- allegation never presented as proven;
- interpretation separated from fact;
- financial content does not become personal advice;
- Bible tradition separated from scripture/historical evidence;
- case reporting separates filing/allegation/ruling;
- script stage blocked while required claims are unsupported;
- source deletion marks claims and downstream stale.

Commit if safe.

---

21. CHECKPOINT J — OUTLINE

Runs only from:

- approved idea;
- approved originality review;
- approved claim map;
- channel profile;
- target duration.

Output:

```
{
  sections: Array<{
    id: string;
```

```
    order: number;
    purpose: string;
    keyPoint: string;
    linkedClaimIds: string[];
    dramaticFunction: string;
    estimatedSeconds: number;
    openLoop?: string;
    proofObjects: string[];
  }>;
}
```

Negative checks:

- all factual sections link claims;
- total duration within target tolerance;
- no claims introduced without claim IDs;
- no scenes or image prompts yet;
- user approval required.

Commit if safe.

---

22. CHECKPOINT K — SCRIPT

Generate section-level script only from approved outline and claims.

Output:

```
{
  sections: Array<{
    id: string;
    outlineSectionId: string;
    purpose: string;
    narration: string;
    estimatedWords: number;
    estimatedSeconds: number;
    linkedClaimIds: string[];
    dramaticFunction: string;
    openLoop?: string;
    visualOpportunities: string[];
    proofObjects: string[];
    retentionRisk: "low" | "medium" | "high";
  }>;
}
```

User can edit and save versions.

Negative checks:

- no unlinked factual claims;
- no unsupported certainty;
- no invented quote;
- no duplicate introduction;
- no excessively repetitive CTA;
- word count aligned to duration;
- no automatic scene generation;
- script version history preserved;
- editing approved script marks all downstream visual/audio/timeline stages stale.

Commit if safe.

---

23. CHECKPOINT L — FACT AND RETENTION REVIEW

Keep these separate review results even if displayed on one screen.

Fact review checks:

- claim linkage;
- unsupported statements;
- qualification loss;
- contradictions;
- risky wording.

Retention review checks:

- hook delay;
- repetitive context;
- unresolved open loops;
- long low-information sections;
- payoff timing;
- CTA placement.

Negative checks:

- fact review cannot rewrite script silently;
- retention review cannot remove required qualifications;
- failed fact review blocks scenes;
- recommendations require explicit apply/save;
- approved script version remains immutable.

Commit if safe.

---

24. CHECKPOINT M — SCENE PLAN

Runs only from approved reviewed script.

A scene is a narrative unit, not necessarily one shot.

Output:

```
{
  scenes: Array<{
    id: string;
    scriptSectionId: string;
    order: number;
    narration: string;
    purpose: string;
    startFrame: number;
    durationFrames: number;
    emotionalState: string;
    proofObject?: string;
    requiredAssets: string[];
  }>;
}
```

Negative checks:

- frame timing is integer;
- scenes do not overlap unexpectedly;
- total scene duration matches script tolerance;
- each scene maps to script;
- no image generation;
- manual edit/reorder supported;
- approval required.

Commit if safe.

---

25. CHECKPOINT N — SHOT PLAN

A scene may contain multiple shots.

Output:

```
{
  shots: Array<{
    id: string;
    sceneId: string;
    order: number;
    startFrame: number;
    durationFrames: number;
    fps: number;
    purpose: string;
    framing: string;
    cameraAngle: string;
    cameraMovement: string;
    subjectAction: string;
    startState: Record<string, unknown>;
    endState: Record<string, unknown>;
    continuityRefs: string[];
  }>;
}
```

Negative checks:

- shot duration > 0;
- shots remain inside scene duration;
- frame order stable;
- no default one-shot-per-scene assumption;
- no impossible camera action;
- no generated asset yet;
- edit/reorder support;
- approval required.

Commit if safe.

---

26. CHECKPOINT O — VISUAL ROUTING

Each approved shot receives a visual mode:

```
reuse
document
diagram
stock_image
stock_video
ai_image
```

```
ai_video
manual_upload
```

Routing should use:

- narrative purpose;
- evidence requirement;
- shot duration;
- approved asset availability;
- continuity needs;
- provider capability.

Negative checks:

- evidence/quote shots should not default to AI image;
- long motion-dependent shots should not default to static image;
- existing approved asset should be considered;
- visual mode must be editable;
- routing is not asset generation;
- approval required before prompt preparation.

Commit if safe.

---

27. CHECKPOINT P — PROMPT PREPARATION

Generate prompts per shot after visual routing approval.

For AI image:

```
{
  shotId: string;
  promptVersionId: string;
  positivePrompt: string;
  negativePrompt: string;
  aspectRatio: string;
  continuityConstraints: string[];
  prohibitedElements: string[];
}
```

Prompt must include:

- subject;
- action;
- setting;

- framing;
- camera;
- lighting;
- mood;
- continuity;
- aspect ratio;
- prohibited content;
- no generated text unless required.

Negative checks:

- no unsupported factual imagery;
- no copyrighted character imitation;
- no accidental logos;
- no duplicated characters;
- no identity drift when continuity applies;
- no prompt from stale shot;
- prompt history preserved;
- prompt approval before generation.

Commit if safe.

---

28. CHECKPOINT Q — ASSET ACQUISITION

Use the verified image model only for `ai_image` shots.

Initially run sequentially or through existing safe queue semantics.

The user has externally limited 9Router usage, so no arbitrary credit limit is required.

Still require:

- persisted job;
- unique input fingerprint;
- idempotency protection;
- explicit shot mapping;
- response parsing;
- secure download/decode;
- file validation;
- local storage;
- asset metadata;
- restart recovery.

Asset states:

```
generated
needs_review
approved
rejected
failed
stale
```

Negative checks:

- HTTP success is not asset success;
- URL alone is not asset success;
- HTML is rejected;
- unsupported MIME rejected;
- corrupted image rejected;
- zero-byte rejected;
- pixel limits checked;
- SHA-256 stored;
- no signed URL persisted;
- no API key in logs;
- failed job does not assign asset;
- duplicate restart does not resend completed request;
- asset maps to the correct shot;
- no auto-approval.

Commit if safe.

---

29. CHECKPOINT R — ASSET REVIEW

User actions:

```
Approve
Reject
Regenerate
Replace with upload
Assign
Unassign
Reuse
View metadata
Open local file
```

Negative checks:

- rejected asset not used in timeline;

- deleting shared asset warns about references;
- changing prompt marks generated asset stale where appropriate;
- regeneration creates new job and asset;
- old versions remain;
- no cross-shot accidental assignment.

Commit if safe.

---

30. CHECKPOINT S — VOICE GENERATION

Only run if TTS capability is verified or local OmniVoice is available.

Generate per script section or logical voice segment.

Do not use synchronous long-running `execFileSync` on Electron main.

Use async process execution with:

- cancellation;
- timeout;
- progress;
- safe stderr;
- output validation.

Validate with FFprobe:

- audio stream exists;
- duration > 0;
- supported codec;
- file opens.

Negative checks:

- empty audio rejected;
- main process must not freeze;
- stale script invalidates voice;
- no automatic voice generation on script save;
- no hidden fallback to another voice provider;
- voice approval required.

Continue other independent work if TTS is blocked.

Commit if safe.

---

31. CHECKPOINT T — SUBTITLE PREPARATION

Create subtitle timing from:

- approved narration segments;
- approved voice timing where available;
- script text.

Negative checks:

- no subtitle extending beyond timeline;
- no overlapping impossible cues;
- preserve punctuation;
- no fabricated text;
- subtitle version tied to script/voice fingerprint;
- edits mark QA stale.

Commit if safe.

---

32. CHECKPOINT U — TIMELINE ASSEMBLY

Build frame-based timeline using approved assets and audio.

Tracks may include:

```
primary_visual
overlay
evidence
voice
music
subtitle
```

Negative checks:

- only approved assets;
- no missing file path;
- no negative frame;
- no zero duration;
- no primary visual gap unless intentional;
- voice timing inside project duration;
- all time stored as integer frames;
- no automatic CapCut export yet.

Commit if safe.

33. CHECKPOINT V — FFMPEG PREVIEW

Use real timeline assets.

Do not produce only a black placeholder video.

Render a short validated preview first.

Then render the current test project preview if safe.

Validate:

- output exists;
- video stream;
- audio stream when expected;
- width/height;
- FPS;
- duration tolerance;
- no unexpected black gaps;
- asset order;
- subtitle timing.

Negative checks:

- missing input file blocks render;
- FFmpeg failure returns safe error;
- no shell interpolation from model output;
- paths escaped safely;
- renderer remains responsive;
- output stored under workspace.

Commit if safe.

---

34. CHECKPOINT W — QA

QA must inspect structured project data and media evidence.

Checks:

- stale upstream artifact;
- missing approval;
- unsupported claim;
- missing shot asset;

- rejected asset on timeline;
- duration mismatch;
- missing audio;
- subtitle overflow;
- continuity issue;
- failed certification;
- broken file path;
- missing CapCut prerequisites.

Negative checks:

- no fake AI QA findings;
- local deterministic checks labeled separately;
- AI-assisted review requires verified text model;
- QA cannot automatically approve project;
- blocking QA issue blocks export.

Commit if safe.

---

35. CHECKPOINT X — CAPCUT DRAFT

Use pycapcut sidecar only after timeline and preview are valid.

First create a compatibility fixture.

Then create editable draft for the current test project.

Validate:

- draft appears in CapCut;
- opens;
- tracks editable;
- visual/audio/subtitle timing preserved;
- not a flattened MP4-only project;
- backup target draft before overwrite.

Negative checks:

- manifest-only output is not success;
- directory existence is not compatibility;
- Python import is not draft verification;
- CapCut draft generation failure is P1;
- do not use unreliable screen-coordinate automation as core export logic.

Continue documentation/audit if CapCut is blocked.

Commit if safe.

---

36. CHECKPOINT Y — PACKAGING EXPORT

Package:

- project metadata;
- approved script;
- references;
- claim map;
- prompts;
- asset manifest;
- preview;
- CapCut draft reference;
- QA report.

Do not include:

- API keys;
- credential references usable outside app;
- signed URLs;
- workspace absolute machine paths;
- unrelated project data.

Negative checks:

- export validation;
- path traversal blocked;
- secret scan;
- missing file reported;
- export is reproducible;
- import remains out of scope unless already safely implemented.

Commit if safe.

---

37. TEST PROJECTS

Use three separate regression projects.

Do not reuse outputs across projects without explicit asset reuse records.

Project A — Bible

```
Name:
Overnight Bible Vertical Slice

Topic:
What did Aaron’s breastpiece symbolize?

Profile:
Bible Mysteries Revealed

Format:
Short

Target:
60–90 seconds
```

Rules:

- separate scripture, historical context, tradition and interpretation;
- do not present interpretation as direct scripture;
- avoid fantasy redesign;
- preserve source qualification.

Project B — Insurance

```
Name:
Overnight Insurance Regression

Topic:
Term life insurance vs whole life insurance

Profile:
Insurance Made Simple

Format:
Short
```

Rules:

- educational only;
- no personalized financial advice;
- no guarantee;
- qualifications preserved.

Project C — Viral Case

```
Name:
Overnight Case Regression

Topic:
An AI copyright dispute involving a creator and a platform

Profile:
Viral Case Files

Format:
Short
```

Rules:

- distinguish allegation, filing and ruling;
- avoid defamation;
- do not invent named individuals;
- preserve uncertainty.

Run Project A as the primary vertical slice.

Use Projects B and C for regression through each implemented stage where inputs are available.

Do not force downstream provider stages when source evidence is missing.

Record per-project status in the overnight report.

---

38. NEGATIVE END-TO-END CHECKS

At every checkpoint verify:

```
No secret leakage
No hidden fixture output
No automatic approval
No Run All
No silent provider fallback
No stale artifact consumption
No failed artifact consumption
No rejected artifact consumption
No downstream automatic execution
No destructive history overwrite
```

```
No duplicate active run
No duplicate provider request on restart
No renderer direct provider request
No renderer arbitrary status mutation
No unvalidated IPC payload
No arbitrary filesystem path
No model-generated code execution
No raw provider HTML shown to user
No raw signed URL persisted
No absolute path in portable export
No stage marked implemented because UI exists
```

Search for dangerous patterns:

```
fixture
mock
demo
return true
Not implemented
TODO
execSync
execFileSync
shell: true
dangerouslySetInnerHTML
webSecurity: false
nodeIntegration: true
Authorization
apiKey
credential
file://
```

Do not blindly remove matches.

Audit each relevant occurrence.

Document legitimate test-only usage separately.

---

39. DATABASE AUDIT

After each migration:

- backup SQLite;
- run migration;

- verify schema;
- verify row counts;
- restart app;
- load old projects;
- create new project;
- inspect foreign keys;
- inspect WAL behavior;
- test rollback.

Never delete old tables or columns during the overnight run unless a safe migration and backup prove no data loss.

Record:

```
migration ID
before schema
after schema
backup path
verification result
rollback result
```

---

40. SECURITY AUDIT

Scan workspace, logs and SQLite for exact secret matches without printing the secret.

Check:

- API key not in SQLite;
- API key not in renderer state;
- API key not in localStorage;
- API key not in project JSON;
- API key not in logs;
- signed URLs redacted;
- IPC schemas validated;
- preload exposes only narrow methods;
- `contextIsolation` remains enabled;
- `nodeIntegration` remains disabled;
- no broad filesystem IPC;
- no untrusted shell execution.

Security regression is P0.

---

41. TEST AND BUILD GATES

After each safe checkpoint run targeted tests.

At major checkpoints run:

```
corepack pnpm lint
corepack pnpm typecheck
corepack pnpm test
corepack pnpm --filter @lsf/desktop build
```

When Electron runtime can be launched, verify actual UI behavior.

Do not claim runtime success based on unit tests alone.

If GUI automation is unavailable:

- launch Electron;
- verify process stability;
- capture screenshots where possible;
- provide exact manual verification steps;
- mark unverified GUI assertions honestly.

---

42. COMMIT POLICY

Create a commit after every safe coherent checkpoint.

Suggested format:

```
feat(workflow): complete reference intake contract
feat(workflow): add transcript cleaning stage
feat(workflow): add competitor DNA stage
feat(workflow): add opportunity map
feat(workflow): add guided idea lab
feat(workflow): add claim map
feat(workflow): add outline and script stages
feat(workflow): add scene and shot planning
feat(media): add approved image acquisition
feat(media): assemble verified preview timeline
feat(capcut): create editable draft bridge
docs(audit): record overnight findings
```

Before each commit:

```
git status
git diff --stat
git diff --check
```

Do not commit:

- workspace media;
- SQLite;
- WAL/SHM;
- API keys;
- `.env`;
- `.venv`;
- logs with secrets;
- CapCut user data;
- temporary test exports;
- generated certification images unless fixtures are explicitly safe and tiny.

Do not amend or rewrite user commits.

Do not push forcefully.

If push credentials are unavailable, retain local commits and record SHAs.

---

43. CONTINUATION RULES

When a stage is blocked:

```
Record why
→ keep dependent stages blocked
→ implement or audit an independent checkpoint
→ continue
```

Examples:

- Transcript Cleaning fails → continue Reference UI audit, provider diagnostics, database tests.
- Image generation fails → continue Voice setup audit, Timeline contract, QA deterministic rules.
- CapCut fails → continue packaging documentation and issue classification.
- One regression project lacks sources → continue other projects.

Do not produce fake downstream outputs.

Do not mark blocked work completed.

---

44. OVERNIGHT END CONDITION

Continue working until one of these occurs:

```
A. All safe planned checkpoints are complete.
B. Only blocked dependent work remains.
C. A P0 condition occurs.
D. Environment prevents further safe work.
```

Do not stop merely because one P1 issue exists.

At the end, update:

```
docs/overnight/OVERNIGHT_RUN_2026-07-30.md
docs/overnight/OPEN_ISSUES_2026-07-30.md
docs/overnight/overnight-issues-2026-07-30.json
docs/overnight/OVERNIGHT_CHECKPOINTS_2026-07-30.md
```

---

45. REQUIRED FINAL REPORT

The final overnight response must contain:

Starting state

- starting SHA;
- branch;
- baseline gates;
- dirty state.

Completed checkpoints

For each:

```
Checkpoint
Implementation status
Runtime status
Commit SHA
```

```
Tests
Manual/runtime evidence
Remaining limitation
```

Use statuses:

```
Not Implemented
UI Only
Local Fixture
Runtime Implemented
Runtime Verified
Blocked
Failed
```

Project matrix

```
| Stage | Bible | Insurance | Viral Case |
|---|---|---|---|
| Reference Intake | | | |
| Validation | | | |
| Transcript Cleaning | | | |
| Segmentation | | | |
| Competitor DNA | | | |
| Opportunity Map | | | |
| Idea Lab | | | |
| Claim Map | | | |
| Outline | | | |
| Script | | | |
| Scene Plan | | | |
| Shot Plan | | | |
| Assets | | | |
| Voice | | | |
| Timeline | | | |
| Preview | | | |
| CapCut | | | |
```

Provider calls

Report actual counts by:

```
provider
model
endpoint
```

```
capability
project
stage
success/failure
```

Never report the key.

Open issues

Summarize P0/P1/P2/P3 counts.

Security result

```
Secret scan
SQLite scan
Log scan
IPC review
Filesystem boundary
```

Database migrations

List migration IDs and backup evidence.

Commits

List SHAs and messages.

Uncommitted files

List honestly.

Recommended morning fix order

Order issues by:

```
P0
P1 blocking earliest workflow stage
P1 blocking multiple projects
P2 correctness
P2 UX
P3
```

---

46. FINAL BEHAVIOR REQUIREMENT

Do not present the application as complete unless a full guided vertical slice has actually passed:

```
Project
→ approved references
→ cleaned transcript
→ approved competitor analysis
→ approved idea
→ approved claims
→ approved script
→ approved shots
→ approved assets
→ approved voice
→ timeline
→ FFmpeg preview
→ QA
→ editable CapCut draft
```

If the vertical slice stops earlier, report the exact highest verified stage.

Do not hide incomplete stages.

Do not replace them with fixture output.

Do not claim that a screen existing means the stage is working.

Begin now.